

CS3002E Compiler Design

Monsoon 2025

Lecture #32

Code Optimization - Control Flow Graph

Department of Computer Science & Engineering
NIT Calicut

CSE: Identifying Redundancies

```
t1=i*4  
t2=a[t1]  
t3=i+1  
i=t3  
t4=i*4  
t5=b[t4]  
t6=t2+t5  
x=t6
```

Is the second occurrence of $i*4$, redundant?

Exercise

Write the 3-address code generated for:

```
if(i<10)
    x=a[i]
else
    x=b[i]
sum=x
```

Generated code:

```
    if(i<10) goto L1

    goto L2
L1:t1=i*4
    t2=a[t1]
    x=t2
    goto L3
L2:t3=i*4
    t4=b[t3]
    x=t4
L3:sum=x
```

Is the second occurrence of $i*4$, redundant?

Identifying redundancies

```
    if(i<10) goto L1
    goto L2
L1:t1=i*4
    t2=a[t1]
    x=t2
    goto L3
L2:t3=i*4
    t4=b[t3]
    x=t4
L3:sum=x
```

Depending on the truth value of $i < 10$, control flows to either L1 or L2.

$i * 4$ is evaluated only once.

The second occurrence of $i * 4$ is not redundant

Exercise

Identify redundant expressions, if any in the following code.

```
    if(i<10) goto L1
    goto L2
L1:t1=i*4
    t2=a[t1]
    x=t2
    goto L3
L2:t3=i*4
    t4=b[t3]
    x=t4
L3:t5=i*4
    c[t5]=x
```

```
    if(i<10) goto L1
    goto L2
L1:t1=i*4
    t2=a[t1]
    x=t2
    goto L3
L2:t3=i*4
    t4=b[t3]
    x=t4
L3:t5=i*4
    c[t5]=x
```

The last occurrence of $i*4$ is redundant.

$i*4$ is evaluated in both the branches of the conditional.

From each branch, control reaches L3.

How to identify such redundancies? How to eliminate?

```
    if(i<10) goto L1
    goto L2
L1:t1=i*4
    t2=a[t1]
    x=t2
    goto L3
L2:t3=i*4
    t4=b[t3]
    x=t4
L3:t5=i*4
    c[t5]=x
```

- Identifying the redundancy requires information regarding the expressions that are computed along the control flow paths reaching L3.
- The information is obtained by doing a *data-flow analysis* of the program.

Data-flow Analysis

- Derives information regarding the flow of data along program execution paths.
- Analysis done in a **Control Flow Graph** representation of the program.
- Example: *Available Expression Analysis* - computes the set of expressions available at every program point. Global CSE requires this information.

Control Flow Graph (or Flow Graph)

- A graph representation of intermediate code.
- The code is partitioned into *basic blocks* such that control flow inside a basic block is sequential.
- The basic blocks are the nodes of the flow graph.
- Graph edges indicate the flow of control between basic blocks.

Basic Blocks

A maximal sequence of consecutive 3-address instructions such that

- flow of control can enter the basic block only through the first instruction in the block
- control leaves the block only after executing the last instruction in the block

Exercise

Identify the basic blocks:

```
L1:t1=i*4
    t2=a[t1]
    x=t2
    goto L3
L2:t3=i*4
    t4=b[t3]
    x=t4
L3:sum=x
```

Basic Blocks

```
t1 = i * 4  
t2 = a[t1]  
x = t2  
goto L3
```

```
t3 = i * 4  
t4 = b[t3]  
x = t4
```

```
sum = x
```

Basic Block: Leader

The first statement in a basic block is known as the *leader* of the Basic Block. A *leader* can be:

- the first instruction in the code
- an instruction that is the target of a conditional or unconditional jump
- instruction that follows a conditional or unconditional jump

Basic Block: Leader

Identify the leaders:

```
    if(i<10) goto L1
    goto L2
L1:t1=i*4
    t2=a[t1]
    x=t2
    goto L3
L2:t3=i*4
    t4=b[t3]
    x=t4
L3:sum=x
```

Basic Block: Leaders

```
    if(i<10) goto L1
    goto L2
L1:t1=i*4
    t2=a[t1]
    x=t2
    goto L3
L2:t3=i*4
    t4=b[t3]
    x=t4
L3:sum=x
```


Basic Blocks

if ($i < 10$) *goto* L1

goto L2

$t1 = i * 4$
 $t2 = a[t1]$
 $x = t2$
goto L3

$t3 = i * 4$
 $t4 = b[t3]$
 $x = t4$

$sum = x$

Control Flow Graph (CFG)

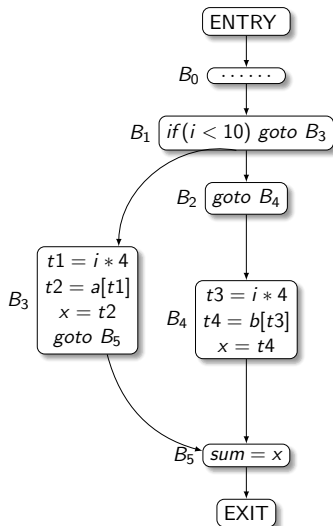
A graph representation of intermediate code.

- each Basic block is a node in the graph
- edge from block B_i to block B_j , if control flows from B_i to B_j

ENTRY and EXIT nodes

- Two empty basic blocks are added to a flow graph:
 - an ENTRY node which represents the starting node of the graph
 - an EXIT node to which all exits out of the graph go.
- An edge is added from ENTRY to the first executable node of the flow graph.
- If a block contains an instruction that could be the last executed instruction in the program, an edge is added from that block to the EXIT node.

Flow Graph for the example code ¹



¹B₀ is assumed to contain the first executable instruction

Exercise

Write the 3-address code generated for the following code fragment:

```
i=0
sum=0
while (i<10)
    sum=sum+i
    i=i+1
x=sum
```

Partition the generated code into basic blocks and draw the flow graph representation. Identify redundant expressions, if any in the code.

References

References:

- Aho A.V., Lam M.S., Sethi R., and Ullman J.D. Compilers: Principles, Techniques, and Tools (ALSU). Pearson Education, 2007.

Further reading:

- ALSU Section 8.4, Chapter 9